

Какой сервер нужен под защиту от подделки лица

Пересчёт конфигурации под реальный масштаб продакшна: через антиспуф проходят АБОНЕНТЫ при реализации газовых баллонов — 60–70 тыс. баллонов/сутки, устойчивая нагрузка 5–15 запросов в секунду (пик — 15–20+). Версия 1 (17.07.2026) считала только инспекторскую нагрузку — единичные проверки, редкие совпадения. Этот сценарий устарел.

Короткий вывод v2: конфигурация из версии 1 (1x T4 без батчинга) держит ~2–5 проверок/сек на GPU-поток — это **в 3–8 раз меньше** требуемых 15–20 запросов/сек. Один GPU без изменения архитектуры сервинга (батчинг) не закрывает вопрос. Нужны: батчинг запросов (Triton/dynamic batching) + минимум 2 GPU в отказоустойчивой схеме.

0. Контекст: что уже было посчитано в версии 1

Версия 1 этого документа (E-GAZ-2-Server-Konfiguraciya-Antispoof.pdf, сохранена без изменений рядом с этим файлом) считала нагрузку от **инспекторов** — предполагалось, что проверки происходят редко и почти никогда не пересекаются по времени. Вывод версии 1: даже 2 одновременные проверки на CPU уже не укладываются в UX-бюджет 5–6 сек, рекомендация — 1× NVIDIA T4 (16 ГБ) с переносом модели на GPU.

Что измерено (факт, CPU i5-11400, 6 ядер/12 потоков)	Время
Детекция лица + landmarks (SCRFD)	ФАКТ ~100–160 мс/кадр
AdaFace IR-101, тёплый прогон	ФАКТ ~342 мс/кадр
AdaFace IR-101, холодный/единичный вызов	ФАКТ ~524 мс/кадр
Passive-PAD	ФАКТ ~20 мс/кадр
Итого на 1 liveness-проверку (4–6 ключевых кадров), CPU, без очереди	ФАКТ 1,85–4,2 с

Критично для пересчёта ниже: сама версия 1 уже честно посчитала предельный случай — «20 проверок одновременно на одном T4 без батчинга» — и получила 6–10 секунд **на всю пачку из 20**, то есть ~0,3–0,5 с на проверку при строго последовательной обработке. Это значит: **один GPU-поток без батчинга держит только ~2-5 проверок/сек** — этот факт уже был в документе, просто не был пересчитан в контексте sustained-нагрузки, потому что о ней тогда не было известно.

НОВОЕ

1. Новый факт от заказчика — что меняется

17.07.2026, после презентации версии 1, Умид ака (заказчик) уточнил: антиспуф — это не только точечная проверка инспектора. Через него проходят **абоненты при реализации газовых баллонов**. Масштаб:

- **60 000–70 000 баллонов/сутки** = столько же прохождений Face ID проверки в день.
- **Устойчивая (sustained) нагрузка: 5–15 запросов/сек** — это не пиковый всплеск, а рабочий поток, который обязан держаться стабильно, не эпизодически.
- Для расчёта конфигурации закладываем **пиковую нагрузку 15–20 запросов/сек** (с запасом над верхней границей sustained), проектный потолок — до 20–25 запросов/сек кратковременно.

Проверка на непротиворечивость (честно, не молчу об этом): 70 000 баллонов за 24 часа — это в среднем 0,81 запрос/сек по всем суткам, а за типичный 10–12-часовой рабочий день — 1,6–1,9 запрос/сек. Заявленные 5–15 запросов/сек устойчиво возможны только если нагрузка сильно неравномерна по времени и/или по точкам продаж — много независимых точек по всей области одновременно бьют в один сервис в рабочие часы, а не размазаны по 24 часам. Это правдоподобно для сети из десятков-сотен точек продаж баллонов, но для честности проектирования: **значения 5–15/15–20 req/s принимаются как исходные данные заказчика**, не выведены нами из 60–70к/сутки арифметически. Если это разные измерения (не одного и того же временного окна) — стоит уточнить у Умида ака паттерн трафика по часам, это влияет на выбор между "запас x2" и "запас x4".

1.1 Почему конфигурация версии 1 не закрывает этот сценарий

Одновременных запросов (req/s, sustained)	Что нужно от инфраструктуры	Версия 1 (1x T4, без батчинга)
1–2 (инспектор)	~0,3–4,2 с/проверка	ОК, укладывается
5–15 (абоненты, sustained)	требуется throughput 5–15 проверок/сек устойчиво	НЕ укладывается — 1 GPU-поток даёт ~2–5/сек
15–20+ (пик)	требуется throughput 15–20+ проверок/сек	Кратный дефицит — нужно в 3–8 раз больше throughput

2. Где на самом деле throughput-потолок — и почему «больше GPU» не единственный рычаг

Ключевое наблюдение, которое меняет расчёт: при batch=1 (одна проверка — один проход через сеть, как считалось в версии 1) GPU T4 использует **около 1% своей реальной вычислительной мощности** на эту задачу.

Прикидка (оценочно): AdaFace IR-101 (ResNet100-класс, ~24 GFLOPs/кадр при 112×112) на 4–6 кадров ≈ 100–150 GFLOPs на проверку. T4 отдаёт до ~65 TFLOPS FP16 (пиково). Если проверка на GPU занимает 0,2–0,5 с (как и оценила версия 1), это ~0,3–0,7 TFLOPS *реально*

использованных — то есть **меньше 1-2% паспортной мощности карты**. Остальное простаивает, потому что batch=1 не даёт GPU параллелить внутри одного вызова.

Отсюда два рычага, ортогональных «купить больше GPU»:

1. **Несколько параллельных inference-воркеров на одной карте** (несколько ONNX Runtime CUDA-сессий/поток) — GPU физически может исполнять несколько независимых проверок одновременно, если сервис не ограничивает себя одним потоком на карту.
2. **Батчинг запросов** (Triton Inference Server / собственная dynamic-batching очередь) — объединяет кадры НЕСКОЛЬКИХ параллельных пользовательских сессий в один batched GPU-вызов (например batch=8-16), что при таких CNN-бэкбонах типично даёт **3-8x прирост throughput** относительно batch=1 (общееинженерная практика для serving ResNet-класса сетей на GPU, а не измерено на этом стеке — см. §5 «Что осталось»).

Без батчинга закрыть 15-20 req/s буквально числом карт — дорого и расточительно: при ~2-5 проверок/сек на один необатченный GPU-поток нужно 4-10 таких потоков (не обязательно 4-10 отдельных карт, но близко к этому без параллелизации), чтобы дотянуть до пика. Батчинг — не косметика, а обязательный элемент архитектуры на этом масштабе, а не последующая оптимизация.

3. Пересчитанный throughput на GPU (оценочно)

Нижне — **оценочные** цифры (нет прямого замера AdaFace IR-101 + SCRFD + PAD в батч-режиме на целевой карте). Порядок величины обоснован общеинженерной практикой инференса ResNet100-класса сетей на T4/A4000/A10 через ONNX Runtime CUDA EP / TensorRT. Финальные цифры — только после нагрузочного теста (см. §5).

Сценарий сервинга	Throughput на 1 GPU (оценочно)	Комментарий
T4, batch=1, один поток (как в версии 1)	ОЦЕНКА 2-5 проверок/сек	подтверждено собственным расчётом версии 1 (20 проверок → 6-10 с)
T4, несколько параллельных воркеров (4-6), без батчинга	ОЦЕНКА 8-15 проверок/сек	рычаг 1, дёшево по деньгам, требует доработки сервинга
T4, dynamic batching (Triton, batch 8-16)	ОЦЕНКА 15-25 проверок/сек	рычаг 2, основной путь к нужному throughput
A4000/A10, dynamic batching	ОЦЕНКА 25-40 проверок/сек	больше «сырой» мощности на карту — меньше карт, больше запас

4. Итоговая рекомендация под 5-15 (пик 15-20) req/s

Конфигурация v2 — под sustained-нагрузку абонентов

Компонент	Рекомендация	Почему
GPU	2× NVIDIA A4000 (16 ГБ) или 2× T4, если A4000 недоступна в аренде	Active-active за балансировщиком. Комбинированный оценочный throughput 30-50 (T4×2) / 50-80 (A4000×2) проверок/сек против пиковых 15-20 — запас ×2-4. При отказе одной карты вторая одна всё ещё держит sustained 5-15 req/s (деградация, не полный отказ) — критично, т.к. сервис теперь блокирует продажу баллонов, а не просто журналирует инспектора.
Сервинг	Triton Inference Server (или эквивалент) с dynamic batching, max_batch_size=16, max_queue_delay≈50-80 мс	Обязательный элемент, не опция — без него throughput на карту падает в 3-8 раз (см. §2-3)
CPU	16-24 vCPU (было 6-8 в v1)	При 15-20 req/s × 4-6 кадров = 75-120 decode/QC/сессионных операций/сек на стороне Python — это уже заметная CPU-нагрузка независимо от GPU
RAM	32-64 ГБ (было 16-32 в v1)	Очереди запросов, буферы батчинга, несколько worker-процессов
Session store	Redis — обязателен, не опционален	При 15-20 req/s сервис физически не проживёт на одном процессе/одном in-memory сторе (см. LIVENESS_CONTRACT_v1.md §4.7) — многоворкерность здесь не «если понадобится масштабирование», а условие входа
Диск	200-300 ГБ NVMe	Больше логов/аудита при таком объёме транзакций в сутки (60-70к)

4.1 Ожидаемый p95 latency (оценочно)

Компонент задержки	Оценка
Ожидание в батч-очереди (queue delay)	50-80 мс
Батчированный inference (4-6 кадров: detect + AdaFace + PAD)	100-300 мс
Оркестрация/декод/сессии (Python-сторона)	50-150 мс
Итого серверная обработка (processing_ms), p95	300-700 мс (оценочно)

Это оставляет UX-бюджету 5-6 секунд (сам бюджет, напомню, ещё не подтверждён владельцем — см. LIVENESS_CONTRACT_v1.md §1) практически весь запас на реальный поворот головы клиентом, а не на вычисления — при условии, что батчинг реализован и очередь не переполняется выше расчётного пика.

4.2 Нужен ли батчинг конкретно вам — короткий ответ

Да, безальтернативно. Без него нужное количество GPU для 15-20 req/s — не 2 карты, а ориентировочно 4-8 (см. §2-3, необатченный поток даёт только 2-5 проверок/сек). Батчинг — это разница между «2 карты» и «до 8 карт» в бюджете аренды ЦОД. Это

самая дорогая строчка в этом пересчёте, и её нельзя пропустить как «оптимизацию на потом».

5. Что осталось — риски и обязательный сбор данных

Все GPU-цифры в этом документе (throughput, latency на T4/A4000/A10) — оценочные, не измеренные. В версии 1 измерения были только на CPU (i5-11400). Ни одного реального прогона AdaFace IR-101 + SCRFD + passive-PAD на целевой GPU-карте, тем более в batched-режиме, не проводилось. Я не занижаю риск: реальный прирост от батчинга может оказаться и меньше 3× (если пайплайн упрётся не в GPU-compute, а в CPU-side decode/QC — тогда нужно больше vCPU, а не GPU), и больше 8× (если TensorRT FP16 даст более агрессивную оптимизацию, чем базовый ONNX Runtime CUDA EP).

- **Нагрузочный тест — обязателен до подписания долгосрочной аренды.** Взять 1× A4000/T4 на пробный период (как и рекомендовала версия 1), поднять Triton с dynamic batching, прогнать синтетическую нагрузку 15–25 req/s с реальными multi-frame сессиями (не единичными кадрами) и замерить фактический p95/throughput. Только после этого финализировать число карт.
- **Redis для session store** — блокер для >1 воркера, что здесь не опция, а условие входа (см. §4). Нужно поднять до первого прод-теста под нагрузкой.
- **Паттерн трафика по часам/точкам продаж** — уточнить у заказчика, насколько 5–15 req/s устойчивы весь рабочий день или это тоже усреднённая/пиковая оценка с своей неопределённостью (см. §1, проверка на непротиворечивость). Это влияет на выбор запаса (×2 против ×4).
- **IDENTITY_MIN, BLINK-порог и APCER/BPCER для multi-frame пайплайна** — те же открытые пункты из LIVENESS_CONTRACT_v1.md §4–5, не относятся к железу, но релевантны для общей production-readiness на этом масштабе (ошибка на 60–70к абонентов/день — это не то же самое по цене, что ошибка на потоке инспекторов).

E-GAZ 2 · Антиспуг-система · Версия 2 от 17.07.2026, подготовлена RZA после уточнения масштаба заказчиком (Умид ака). Дополняет, не заменяет версию 1 от 17.07.2026 (сохранена как есть). Все throughput/latency-цифры на GPU помечены как ОЦЕНКА и требуют нагрузочного теста перед финальным подтверждением конфигурации и подпиской на долгосрочную аренду.